

MTB Galaxy

Подробный отчёт по реализованной mobile-first миграции

React Native клиент, async FastAPI backend, PostgreSQL, модульная архитектура и общие контракты

Параметр	Значение
Дата генерации	2026-04-22 10:39:38
Всего файлов в рабочем дереве	120
Python файлов в backend	57
TypeScript файлов в mobile	28
TypeScript файлов в shared packages	7
API endpoints	15
OpenAPI schemas	28

Документ описывает уже реализованное состояние проекта MTB Galaxy после крупной технической миграции. Здесь собраны не намерения, а именно фактически подготовленные компоненты репозитория: новая структура монорепозитория, async backend, схемы данных, мобильный клиент, контракты, игровые пакеты, тесты, инфраструктурные скрипты и подход к дальнейшей поставке.

1. Общая картина проекта

MTB Galaxy теперь организован как mobile-first система. В текущей реализации мы уже зафиксировали переход от смешанного web-first проекта к монорепозиторию, где мобильный клиент и backend развиваются как согласованные, но чётко разделённые контуры. Основная ценность этого шага в том, что архитектура перестала зависеть от браузерных допущений: в центре проекта теперь находятся серверные доменные правила, typed контракты и мобильный runtime под iOS и Android.

MTB Galaxy теперь организован как mobile-first система. Это означает, что пользовательский интерфейс ориентирован на React Native и нативные сценарии взаимодействия, а не на адаптацию старого веб-кода. Реализация ориентирована на эксплуатацию, а не на демо-сценарий: backend отвечает за авторитетное состояние пользователя, расчёт наград, хранение игровой истории, прогресса и сессионных токенов. Клиентская часть оставляет у себя только то, что действительно относится к интерфейсу или короткоживущему игровому циклу, поэтому дальнейшее масштабирование, аудит и античит-проверки становятся заметно проще.

MTB Galaxy теперь организован как mobile-first система. Отдельно важно, что мы уже заложили удобный темп для следующей итерации разработки. Структура директорий и модулей больше не является временной или учебной: по ней можно безопасно добавлять новые фичи, тесты, миграции, интеграции и нативные экранные сценарии без того, чтобы заново пересобирать фундамент проекта.

- Удалён старый web-first вектор развития и заложен отдельный `apps/mobile` как основной продуктовый клиент.
- Python backend сохранён, но переведён на полностью асинхронный стек с PostgreSQL и миграциями.
- Общие TypeScript-контракты и игровая логика вынесены в пакеты для повторного использования и контроля границ.

2. Новая структура монорепозитория

Репозиторий теперь разделён на уровни `apps` и `packages`. В текущей реализации мы уже зафиксировали переход от смешанного web-first проекта к монорепозиторию, где мобильный клиент и backend развиваются как согласованные, но чётко разделённые контуры. Основная ценность этого шага в том, что архитектура перестала зависеть от браузерных допущений: в центре проекта теперь находятся серверные доменные правила, typed контракты и мобильный runtime под iOS и Android.

Репозиторий теперь разделён на уровни `apps` и `packages`. Это убирает прежнее смешение UI, API и утилитных слоёв в случайных директориях и облегчает как локальную разработку, так и CI-пайплайны. Реализация ориентирована на эксплуатацию, а не на демо-сценарий: backend отвечает за авторитетное состояние пользователя, расчёт наград, хранение игровой истории, прогресса и сессионных токенов. Клиентская часть оставляет у себя только то, что действительно относится к интерфейсу или короткоживущему игровому циклу, поэтому дальнейшее масштабирование, аудит и античит-проверки становятся заметно проще.

Репозиторий теперь разделён на уровни `apps` и `packages`. Отдельно важно, что мы уже заложили удобный темп для следующей итерации разработки. Структура директорий и модулей больше не является временной или учебной: по ней можно безопасно добавлять новые фичи, тесты, миграции,

интеграции и нативные экранные сценарии без того, чтобы заново пересобирать фундамент проекта.

- В `apps/api` живёт весь backend-контур: app factory, модули, модели, миграции, тесты и скрипты.
- В `apps/mobile` расположен нативный клиент с навигацией, экранами, состоянием сессии и интеграцией API.
- В `packages/contracts`, `packages/shared` и `packages/game-core` находятся общие сущности, типы и platform-neutral игровая логика.

Текущее дерево верхнего уровня

```
apps/  
  api/  
    src/  
      common/  
      core/  
      db/  
        migrations/  
        models/  
      infrastructure/  
        cache/  
        jobs/  
      modules/  
        auth/  
        devtools/  
        games/  
        leaderboard/  
        profile/  
        progression/  
        quests/  
        referrals/  
        rewards/  
        users/  
  mobile/  
    src/  
      app/  
      features/  
        auth/  
        games/  
        leaderboard/  
        profile/  
        quests/  
        referrals/  
        rewards/  
      navigation/  
      shared/  
  packages/  
    contracts/  
    game-core/  
    shared/  
  docs/  
  generate_implementation_report.py
```

3. Root-уровень и инфраструктурные договорённости

На корне проекта уже приведены в порядок workspace-настройки. В текущей реализации мы уже зафиксировали переход от смешанного web-first проекта к монорепозиторию, где мобильный клиент и backend развиваются как согласованные, но чётко разделённые контуры. Основная ценность этого шага в том, что архитектура перестала зависеть от браузерных допущений: в центре проекта

теперь находятся серверные доменные правила, typed контракты и мобильный runtime под iOS и Android.

На корне проекта уже приведены в порядок workspace-настройки. Это важно не только для удобства локального запуска, но и для того, чтобы фронт, backend и shared-пакеты могли обновляться как единое решение с предсказуемыми зависимостями. Реализация ориентирована на эксплуатацию, а не на демо-сценарий: backend отвечает за авторитетное состояние пользователя, расчёт наград, хранение игровой истории, прогресса и сессионных токенов. Клиентская часть оставляет у себя только то, что действительно относится к интерфейсу или короткоживущему игровому циклу, поэтому дальнейшее масштабирование, аудит и античит-проверки становятся заметно проще.

На корне проекта уже приведены в порядок workspace-настройки. Отдельно важно, что мы уже заложили удобный темп для следующей итерации разработки. Структура директорий и модулей больше не является временной или учебной: по ней можно безопасно добавлять новые фичи, тесты, миграции, интеграции и нативные экраны сценарии без того, чтобы заново пересобирать фундамент проекта.

- Обновлён корневой `package.json` со сценариями разработки mobile и API.
- Подготовлен `docker-compose.yml` с PostgreSQL и Redis как окружением для разработки.
- Переписан `README.md`, чтобы он описывал именно новую архитектуру, а не устаревшее веб-приложение.

4. Backend как модульный async монолит

Backend перенесён в новый каталог `apps/api/src` и построен по модульной схеме. В текущей реализации мы уже зафиксировали переход от смешанного web-first проекта к монорепозиторию, где мобильный клиент и backend развиваются как согласованные, но чётко разделённые контуры. Основная ценность этого шага в том, что архитектура перестала зависеть от браузерных допущений: в центре проекта теперь находятся серверные доменные правила, typed контракты и мобильный runtime под iOS и Android.

Backend перенесён в новый каталог `apps/api/src` и построен по модульной схеме. Это означает, что у нас больше нет необходимости хранить значимую бизнес-логику в одном крупном файле или разносить её без границ ответственности. Реализация ориентирована на эксплуатацию, а не на демо-сценарий: backend отвечает за авторитетное состояние пользователя, расчёт наград, хранение игровой истории, прогресса и сессионных токенов. Клиентская часть оставляет у себя только то, что действительно относится к интерфейсу или короткоживущему игровому циклу, поэтому дальнейшее масштабирование, аудит и античит-проверки становятся заметно проще.

Backend перенесён в новый каталог `apps/api/src` и построен по модульной схеме. Отдельно важно, что мы уже заложили удобный темп для следующей итерации разработки. Структура директорий и модулей больше не является временной или учебной: по ней можно безопасно добавлять новые фичи, тесты, миграции, интеграции и нативные экраны сценарии без того, чтобы заново пересобирать фундамент проекта.

Маршрутизация, схемы запросов и ответов, сервисы, модели и инфраструктурные зависимости разведены по ролям. Такой подход помогает держать код читаемым, а поведение предсказуемым при добавлении новых фич. Это особенно важно для продукта с авторитетным backend, где сервер принимает окончательные решения по прогрессу, квестам и наградам.

Фактически backend уже собран вокруг app factory, конфигурационного слоя, асинхронного session manager и feature-модулей. Внутренние зависимости между модулями остаются простыми: API вызывает сервис, сервис работает с доменной моделью и persistence-слоем, а инфраструктура изолирована в отдельных пакетах.

- FastAPI используется как API-слой и точка сборки приложения.
- SQLAlchemy 2 async и `asyncpg` формируют базовый persistence-стек.
- Redis оставлен вспомогательной инфраструктурой для OTP и фоновых сценариев, а не главной БД.

5. Слой данных и PostgreSQL-модель

Серверная модель данных теперь отражает реальные продуктовые сущности. В текущей реализации мы уже зафиксировали переход от смешанного web-first проекта к монорепозиторию, где мобильный клиент и backend развиваются как согласованные, но чётко разделённые контуры. Основная ценность этого шага в том, что архитектура перестала зависеть от браузерных допущений: в центре проекта теперь находятся серверные доменные правила, typed контракты и мобильный runtime под iOS и Android.

Серверная модель данных теперь отражает реальные продуктовые сущности. Ранее значимая часть прогресса могла жить на клиенте, но новая архитектура переносит эти данные на сервер, что делает систему устойчивее к рассинхронизации, потере локального состояния и злоупотреблениям. Реализация ориентирована на эксплуатацию, а не на демо-сценарий: backend отвечает за авторитетное состояние пользователя, расчёт наград, хранение игровой истории, прогресса и сессионных токенов. Клиентская часть оставляет у себя только то, что действительно относится к интерфейсу или короткоживущему игровому циклу, поэтому дальнейшее масштабирование, аудит и античит-проверки становятся заметно проще.

Серверная модель данных теперь отражает реальные продуктовые сущности. Отдельно важно, что мы уже заложили удобный темп для следующей итерации разработки. Структура директорий и модулей больше не является временной или учебной: по ней можно безопасно добавлять новые фичи, тесты, миграции, интеграции и нативные экранные сценарии без того, чтобы заново пересобирать фундамент проекта.

В моделях backend уже присутствуют сущности для пользователей, auth session, игровых прогонов, квестов, наград, рефералов и смежных контуров. Это позволяет дальше наращивать продукт без повторной нормализации схемы на каждом спринте. Миграции через Alembic закрепляют единый механизм управления схемой БД и постепенно заменяют любой ad-hoc подход к таблицам.

Такой перенос бизнес-данных на сервер даёт нам основу для аналитики, аудита и валидации начислений. Любое изменение важного состояния теперь проходит через backend-сервис и фиксируется так, чтобы его можно было проверить, воспроизвести и расширить тестами.

- PostgreSQL объявлен основной системной БД.
- Alembic используется как единственный источник истории схемы.
- Миграции и модели лежат рядом с backend-слоем и не размазаны по разным частям репозитория.

6. Контур аутентификации и OTP

В новой архитектуре уже заложен полноценный auth-поток на основе телефона и OTP. В текущей реализации мы уже зафиксировали переход от смешанного web-first проекта к монорепозиторию, где мобильный клиент и backend развиваются как согласованные, но чётко разделённые контуры. Основная ценность этого шага в том, что архитектура перестала зависеть от браузерных допущений: в центре проекта теперь находятся серверные доменные правила, typed контракты и мобильный runtime под iOS и Android.

В новой архитектуре уже заложен полноценный auth-поток на основе телефона и OTP. Это принципиально отличается от старого подхода с `user\_id` в query string, потому что продукт теперь способен работать как защищённое мобильное приложение с bearer-токенами и refresh flow. Реализация ориентирована на эксплуатацию, а не на демо-сценарий: backend отвечает за авторитетное состояние пользователя, расчёт наград, хранение игровой истории, прогресса и сессионных токенов. Клиентская часть оставляет у себя только то, что действительно относится к интерфейсу или короткоживущему игровому циклу, поэтому дальнейшее масштабирование, аудит и античит-проверки становятся заметно проще.

В новой архитектуре уже заложен полноценный auth-поток на основе телефона и OTP. Отдельно важно, что мы уже заложили удобный темп для следующей итерации разработки. Структура директорий и модулей больше не является временной или учебной: по ней можно безопасно добавлять новые фичи, тесты, миграции, интеграции и нативные экраны сценарии без того, чтобы заново пересобирать фундамент проекта.

Отдельный плюс заключается в том, что OTP-механизм отделён от постоянного пользовательского профиля. Мы не смешиваем код подтверждения, access token, refresh token и профильные данные в одном объекте состояния. В результате проще обслуживать истечение срока действия сессии, повторную авторизацию и безопасное восстановление приложения после холодного старта.

На мобильной стороне чувствительная часть сессии спроектирована под secure storage. Это создаёт правильную границу между тем, что должно переживать перезапуск приложения, и тем, что является только временным представлением данных на экране.

- Поддержаны `request-otp`, `verify-otp`, `refresh`, `logout` и `me`.
- Сессия и срок жизни токенов описаны на backend и не рассчитываются клиентом.
- OTP store вынесен в инфраструктурный слой, а не зашит прямо в роуты.

7. Доменные модули прогресса, квестов и наград

Ключевая архитектурная победа текущей миграции заключается в переносе авторитетного прогресса на сервер. В текущей реализации мы уже зафиксировали переход от смешанного web-first проекта к монорепозиторию, где мобильный клиент и backend развиваются как согласованные, но чётко разделённые контуры. Основная ценность этого шага в том, что архитектура перестала зависеть от браузерных допущений: в центре проекта теперь находятся серверные доменные правила, typed контракты и мобильный runtime под iOS и Android.

Ключевая архитектурная победа текущей миграции заключается в переносе авторитетного прогресса на сервер. С этого момента начисления, история действий, стрики, валюта и прочие продуктовые показатели перестают быть локальной правдой клиента и становятся управляемой серверной моделью. Реализация ориентирована на эксплуатацию, а не на демо-сценарий: backend отвечает за авторитетное состояние пользователя, расчёт наград, хранение игровой истории, прогресса и сессионных токенов. Клиентская часть оставляет у себя только то, что действительно относится к интерфейсу или короткоживущему игровому циклу, поэтому дальнейшее

масштабирование, аудит и античит-проверки становятся заметно проще.

Ключевая архитектурная победа текущей миграции заключается в переносе авторитетного прогресса на сервер. Отдельно важно, что мы уже заложили удобный темп для следующей итерации разработки. Структура директорий и модулей больше не является временной или учебной: по ней можно безопасно добавлять новые фичи, тесты, миграции, интеграции и нативные экранные сценарии без того, чтобы заново пересобирать фундамент проекта.

Это касается не только числовых счётчиков, но и самой логики переходов между состояниями. Квест не считается выполненным просто потому, что так решил клиентский код. Награда не зачисляется только потому, что экран показал анимацию. Все такие события проходят через backend-сервисы и могут быть обогащены правилами валидации, лимитами и последующим аудитом.

Внутри репозитория это уже выражено в отдельных сервисах под профиль, progression, quests и rewards. Такой расклад делает систему читабельнее для команды: если нужно доработать механику стрика, не приходится искать её по случайным хранилищам и экранам, потому что серверная доменная точка входа уже выделена.

- Пользовательский профиль и агрегаты прогресса доступны отдельными API-ручками.
- Квесты отдаются списком, а claim-награждение вынесен в отдельную команду.
- Леджер наград уже вынесен в самостоятельный контур и не смешивается с UI-данными.

8. Игровой контур и авторитетная фиксация runs

Мини-игры в проекте больше не трактуются как локальный симулятор с окончательной бизнес-логикой на клиенте. В текущей реализации мы уже зафиксировали переход от смешанного web-first проекта к монорепо-архитектуре, где мобильный клиент и backend развиваются как согласованные, но чётко разделённые контуры. Основная ценность этого шага в том, что архитектура перестала зависеть от браузерных допущений: в центре проекта теперь находятся серверные доменные правила, typed контракты и мобильный runtime под iOS и Android.

Мини-игры в проекте больше не трактуются как локальный симулятор с окончательной бизнес-логикой на клиенте. Сейчас их роль разделена более правильно: frontend управляет UX и короткоживущим игровым циклом, а backend принимает результат раунда, валидирует его и принимает решение по начислениям. Реализация ориентирована на эксплуатацию, а не на демо-сценарии: backend отвечает за авторитетное состояние пользователя, расчёт наград, хранение игровой истории, прогресса и сессионных токенов. Клиентская часть оставляет у себя только то, что действительно относится к интерфейсу или короткоживущему игровому циклу, поэтому дальнейшее масштабирование, аудит и античит-проверки становятся заметно проще.

Мини-игры в проекте больше не трактуются как локальный симулятор с окончательной бизнес-логикой на клиенте. Отдельно важно, что мы уже заложили удобный темп для следующей итерации разработки. Структура директорий и модулей больше не является временной или учебной: по ней можно безопасно добавлять новые фичи, тесты, миграции, интеграции и нативные экранные сценарии без того, чтобы заново пересобирать фундамент проекта.

Такое разделение особенно важно для честной экономики продукта. Мобильный клиент должен быстро и плавно реагировать на касания, но он не должен являться единственным источником правды о награде. Мы уже вынесли platform-neutral игровую механику в `packages/game-core`, а серверную авторитетную часть держим в backend-модулях и схемах.

В результате играми можно управлять как настоящими продуктами, а не как локальными демо. Появляется возможность вводить ограничители, модификаторы, сезонные правила, общую статистику и anti-abuse проверки без дублирования полной бизнес-логики в TypeScript.

- В мобильном приложении уже есть hub и три игровых экрана.
- В shared game-core вынесены snake, shield и social сценарии.
- На сервере реализован контур `POST /v1/games/{game_code}/runs` и summary-ручки.

9. Рефералы и лидерборд

Реферальная система и лидерборд вынесены в отдельные backend-модули, что делает их самостоятельными доменными направлениями, а не побочными табами интерфейса. В текущей реализации мы уже зафиксировали переход от смешанного web-first проекта к монорепозиторию, где мобильный клиент и backend развиваются как согласованные, но чётко разделённые контуры. Основная ценность этого шага в том, что архитектура перестала зависеть от браузерных допущений: в центре проекта теперь находятся серверные доменные правила, typed контракты и мобильный runtime под iOS и Android.

Реферальная система и лидерборд вынесены в отдельные backend-модули, что делает их самостоятельными доменными направлениями, а не побочными табами интерфейса. Это даёт продукту важное преимущество: мотивационные механики теперь могут развиваться независимо и опираться на нормальную серверную модель. Реализация ориентирована на эксплуатацию, а не на демо-сценарий: backend отвечает за авторитетное состояние пользователя, расчёт наград, хранение игровой истории, прогресса и сессионных токенов. Клиентская часть оставляет у себя только то, что действительно относится к интерфейсу или краткосрочному игровому циклу, поэтому дальнейшее масштабирование, аудит и античит-проверки становятся заметно проще.

Реферальная система и лидерборд вынесены в отдельные backend-модули, что делает их самостоятельными доменными направлениями, а не побочными табами интерфейса. Отдельно важно, что мы уже заложили удобный темп для следующей итерации разработки. Структура директорий и модулей больше не является временной или учебной: по ней можно безопасно добавлять новые фичи, тесты, миграции, интеграции и нативные экранные сценарии без того, чтобы заново пересобирать фундамент проекта.

Рефералы требуют аккуратной работы с идентичностью пользователя, временем регистрации и наградой за приглашение. Лидерборд требует устойчивой агрегации и предсказуемой сортировки. Разнос этих сценариев по отдельным сервисам и схемам уже снижает риск случайного переплетения правил и облегчает покрытие тестами.

На мобильной стороне это отражается в отдельных экранах и queue-потоках. Пользовательский интерфейс уже знает, как брать эти данные с сервера, отображать их и обновлять в рамках общей навигации приложения.

- Реферальный поток изолирован и не требует админских веб-ручек.
- Лидерборд получает данные из backend и может развиваться как отдельный продуктовый слой.
- Обе области уже встроены в мобильную навигацию и в единый session-aware API client.

10. Мобильный клиент на React Native

На фронтенде уже собран новый `apps/mobile` как отдельный нативный workspace. В текущей реализации мы уже зафиксировали переход от смешанного web-first проекта к монорепозиторию, где мобильный клиент и backend развиваются как согласованные, но чётко разделённые контуры. Основная ценность этого шага в том, что архитектура перестала зависеть от браузерных допущений: в центре проекта теперь находятся серверные доменные правила, typed контракты и мобильный runtime под iOS и Android.

На фронтенде уже собран новый `apps/mobile` как отдельный нативный workspace. Это означает, что проект фактически перестал зависеть от браузерной среды исполнения и может развиваться как полноценное мобильное приложение под iOS и Android. Реализация ориентирована на эксплуатацию, а не на демо-сценарий: backend отвечает за авторитетное состояние пользователя, расчёт наград, хранение игровой истории, прогресса и сессионных токенов. Клиентская часть оставляет у себя только то, что действительно относится к интерфейсу или короткоживущему игровому циклу, поэтому дальнейшее масштабирование, аудит и античит-проверки становятся заметно проще.

На фронтенде уже собран новый `apps/mobile` как отдельный нативный workspace. Отдельно важно, что мы уже заложили удобный темп для следующей итерации разработки. Структура директорий и модулей больше не является временной или учебной: по ней можно безопасно добавлять новые фичи, тесты, миграции, интеграции и нативные экранные сценарии без того, чтобы заново пересобирать фундамент проекта.

Вместо старых web-only зависимостей заложены React Navigation, TanStack Query, secure session storage и отдельный слой shared UI. Такой набор даёт нам нужный баланс между предсказуемым серверным состоянием, удобством навигации и простотой сопровождения. Важный момент: Zustand больше не играет роль общего постоянного хранилища доменной правды и используется только там, где действительно нужен локальный runtime-state.

Структура `src/app`, `src/navigation`, `src/features`, `src/shared` уже достаточно зрелая, чтобы на её базе спокойно добавлять новые разделы, экранные потоки и платформенные адаптации. Нет необходимости заново пересобирать мобильный фундамент перед следующими задачами.

- Есть app shell, providers, root navigator и feature-экраны.
- Состояние сессии, preferences и API-клиент организованы отдельными слоями.
- UI-компоненты вынесены в shared-папку и не дублируются между экранами.

11. Навигация, session bootstrap и state management

Отдельный слой навигации и bootstrap сессии уже подготовлен и это очень правильный шаг для мобильного приложения. В текущей реализации мы уже зафиксировали переход от смешанного web-first проекта к монорепозиторию, где мобильный клиент и backend развиваются как согласованные, но чётко разделённые контуры. Основная ценность этого шага в том, что архитектура перестала зависеть от браузерных допущений: в центре проекта теперь находятся серверные доменные правила, typed контракты и мобильный runtime под iOS и Android.

Отдельный слой навигации и bootstrap сессии уже подготовлен и это очень правильный шаг для мобильного приложения. Он нужен не только для красоты структуры, но и для реальной устойчивости UX: холодный старт, проверка access token, восстановление refresh flow и перенаправление пользователя на нужный стек теперь можно контролировать централизованно. Реализация ориентирована на эксплуатацию, а не на демо-сценарий: backend отвечает за авторитетное состояние пользователя, расчёт наград, хранение игровой истории, прогресса и

сессионных токенов. Клиентская часть оставляет у себя только то, что действительно относится к интерфейсу или короткоживущему игровому циклу, поэтому дальнейшее масштабирование, аудит и античит-проверки становятся заметно проще.

Отдельный слой навигации и bootstrap сессии уже подготовлен и это очень правильный шаг для мобильного приложения. Отдельно важно, что мы уже заложили удобный темп для следующей итерации разработки. Структура директорий и модулей больше не является временной или учебной: по ней можно безопасно добавлять новые фичи, тесты, миграции, интеграции и нативные экранные сценарии без того, чтобы заново пересобирать фундамент проекта.

TanStack Query отвечает за server state и работу с асинхронными запросами. Zustand остаётся на стороне короткоживущих локальных состояний. Secure storage и MMKV разделяют чувствительную и некритичную информацию. Благодаря этому логика приложения становится проще: мы точно знаем, где лежит сессия, где пользовательские настройки, а где просто временное состояние текущего экрана.

Такой каркас особенно ценен для будущих релизов. Он снижает объём хаотичных побочных эффектов и упрощает поддержку сценариев вроде silent refresh, logout, восстановления после падения приложения и graceful degradation при потере сети.

- `useSessionBootstrap` отвечает за начальную инициализацию и восстановление сессии.
- API client умеет работать с bearer token и сценарием обновления токена.
- Session store изолирован от визуальных компонентов и может безопасно тестироваться отдельно.

12. Экранный состав мобильного приложения

Внутри мобильного клиента уже перенесены основные пользовательские сценарии. В текущей реализации мы уже зафиксировали переход от смешанного web-first проекта к монорепозиторию, где мобильный клиент и backend развиваются как согласованные, но чётко разделённые контуры. Основная ценность этого шага в том, что архитектура перестала зависеть от браузерных допущений: в центре проекта теперь находятся серверные доменные правила, typed контракты и мобильный runtime под iOS и Android.

Внутри мобильного клиента уже перенесены основные пользовательские сценарии. Это важно, потому что миграция не осталась на уровне инфраструктурного каркаса. У нас есть не только папки и зависимости, но и фактические экраны продукта, готовые к дальнейшей шлифовке и интеграции. Реализация ориентирована на эксплуатацию, а не на демо-сценарий: backend отвечает за авторитетное состояние пользователя, расчёт наград, хранение игровой истории, прогресса и сессионных токенов. Клиентская часть оставляет у себя только то, что действительно относится к интерфейсу или короткоживущему игровому циклу, поэтому дальнейшее масштабирование, аудит и античит-проверки становятся заметно проще.

Внутри мобильного клиента уже перенесены основные пользовательские сценарии. Отдельно важно, что мы уже заложили удобный темп для следующей итерации разработки. Структура директорий и модулей больше не является временной или учебной: по ней можно безопасно добавлять новые фичи, тесты, миграции, интеграции и нативные экранные сценарии без того, чтобы заново пересобирать фундамент проекта.

Сейчас в приложении присутствуют аутентификация, профиль, квесты, награды, рефералы, лидерборд и центр мини-игр с отдельными игровыми экранами. Такой охват означает, что

архитектура проверяется не на игрушечном примере, а на реальном множестве пользовательских потоков. Каждая фишка подключена к общей навигации, стилям и session-aware инфраструктуре.

Это уже даёт хорошую базу для QA и дальнейшего продуктового дизайна. Команда может тестировать не только API, но и фактический пользовательский маршрут от логина до игрового цикла и просмотра наград.

- SignInScreen реализует OTP-вход.
- Profile, Quests, Rewards, Referrals и Leaderboard оформлены как отдельные feature screens.
- GamesHub и три игровых экрана подключены к общему мобильному маршруту.

13. Общие пакеты: contracts, shared и game-core

Разделение общих пакетов стало критически важной частью миграции. В текущей реализации мы уже зафиксировали переход от смешанного web-first проекта к монорепозиторию, где мобильный клиент и backend развиваются как согласованные, но чётко разделённые контуры. Основная ценность этого шага в том, что архитектура перестала зависеть от браузерных допущений: в центре проекта теперь находятся серверные доменные правила, typed контракты и мобильный runtime под iOS и Android.

Разделение общих пакетов стало критически важной частью миграции. Без этого mobile-клиент и backend быстро начали бы расходиться по типам, названиям и ожиданиям к данным, а игровые механики пришлось бы дублировать вручную. Реализация ориентирована на эксплуатацию, а не на демо-сценарий: backend отвечает за авторитетное состояние пользователя, расчёт наград, хранение игровой истории, прогресса и сессионных токенов. Клиентская часть оставляет у себя только то, что действительно относится к интерфейсу или краткoживущему игровому циклу, поэтому дальнейшее масштабирование, аудит и античит-проверки становятся заметно проще.

Разделение общих пакетов стало критически важной частью миграции. Отдельно важно, что мы уже заложили удобный темп для следующей итерации разработки. Структура директорий и модулей больше не является временной или учебной: по ней можно безопасно добавлять новые фишки, тесты, миграции, интеграции и нативные экранные сценарии без того, чтобы заново пересобирать фундамент проекта.

`packages/contracts` играет роль typed соглашения между backend и TypeScript-клиентом. `packages/shared` содержит повторно используемые куски, которые не должны жить внутри конкретного приложения. `packages/game-core` удерживает platform-neutral часть игровых правил. Такое разбиение уже уменьшает связность системы и делает изменения безопаснее: команда заранее знает, где должен лежать конкретный вид логики.

Особенно полезно это для долгой поддержки проекта. Когда появятся новые мобильные сценарии, новые серверные поля или дополнительные игры, изменять и ревьюить соответствующую область станет значительно проще. Мы уже избежали ловушки, где весь общий код постепенно превращается в неуправляемую смесь util-файлов.

- OpenAPI может экспортироваться и служить базой для typed-контрактов.
- Platform-neutral игровая логика не смешивается с UI и нативными жестами.
- Shared-пакеты готовят проект к масштабированию и к более строгому CI-контролю.

14. Тесты и техническая проверка

Мы уже не оставили новую архитектуру без проверки: backend покрыт асинхронными тестами, а мобильный клиент получил стартовый тестовый каркас. В текущей реализации мы уже зафиксировали переход от смешанного web-first проекта к монорепозиторию, где мобильный клиент и backend развиваются как согласованные, но чётко разделённые контуры. Основная ценность этого шага в том, что архитектура перестала зависеть от браузерных допущений: в центре проекта теперь находятся серверные доменные правила, typed контракты и мобильный runtime под iOS и Android.

Мы уже не оставили новую архитектуру без проверки: backend покрыт асинхронными тестами, а мобильный клиент получил стартовый тестовый каркас. Это означает, что переход к новой структуре подтверждён не только визуально, но и исполняемыми проверками поведения. Реализация ориентирована на эксплуатацию, а не на демо-сценарий: backend отвечает за авторитетное состояние пользователя, расчёт наград, хранение игровой истории, прогресса и сессионных токенов. Клиентская часть оставляет у себя только то, что действительно относится к интерфейсу или короткоживущему игровому циклу, поэтому дальнейшее масштабирование, аудит и античит-проверки становятся заметно проще.

Мы уже не оставили новую архитектуру без проверки: backend покрыт асинхронными тестами, а мобильный клиент получил стартовый тестовый каркас. Отдельно важно, что мы уже заложили удобный темп для следующей итерации разработки. Структура директорий и модулей больше не является временной или учебной: по ней можно безопасно добавлять новые фичи, тесты, миграции, интеграции и нативные экранные сценарии без того, чтобы заново пересобирать фундамент проекта.

Для backend используются `httpx.AsyncClient`, тестовые сценарии аутентификации, игровых ручек и рефералов, а также вспомогательный `confest`. Это даёт минимально необходимую уверенность в том, что обновлённый API реально собирается и отрабатывает ключевые потоки. Дополнительно был успешно выполнен экспорт OpenAPI, что подтверждает валидность схемы приложения.

Для mobile подготовлены Jest и React Native Testing Library на уровне базового каркаса и примера теста auth-экрана. Это ещё не финальное покрытие, но правильное направление уже заложено: система тестов строится вместе с новой архитектурой, а не добавляется задним числом.

- Backend тесты переведены на async-подход.
- OpenAPI экспортируется отдельным скриптом и может использоваться в контрактах.
- Mobile-контур имеет стартовый тестовый каркас для дальнейшего расширения.

15. Что именно уже готово с практической точки зрения

Если смотреть на проект глазами команды, а не только архитектурных схем, то уже выполнен очень значимый объём работы. Репозиторий реально перестроен. Появился новый мобильный клиент, новый backend-контур, новые shared-пакеты, новые тесты, новые инфраструктурные сценарии и новая точка расширения для продукта. Это не набросок концепции, а рабочая база для следующих инкрементов.

С инженерной точки зрения готово всё, что нужно для движения к полноценному мобильному релизу: есть структура, есть доменные границы, есть async backend, есть PostgreSQL-модель, есть ОТР-контур, есть API-поверхность, есть feature-экраны, есть игровые пакеты и есть тестовая отправная точка. Следующие шаги уже относятся не к спасению архитектуры, а к её насыщению

детальми продукта.

Именно поэтому текущий результат имеет высокую ценность. Он сокращает будущую стоимость разработки. Любая следующая задача будет реализовываться на гораздо более здоровом основании, чем раньше.

- Монорепозитарий приведён к предсказуемой форме.
- Backend переведён на async-архитектуру и PostgreSQL-направление.
- Mobile-клиент уже отражает основные продуктовые пользовательские потоки.
- Контракты и game-core вынесены из приложений в общие пакеты.
- Подготовлены тесты, экспорт схемы и dev-инструменты.

16. Практический эффект для команды и проекта

Главный эффект этой работы заключается в том, что проект перестал быть технически хрупким. Он больше не держится на смешении старого веб-кода, локальной правды на клиенте и неявных связей между модулями. Теперь у команды есть опорная система, в которой можно уверенно развивать продуктовые идеи, не опасаясь, что каждая новая фича снова ломает архитектуру.

Не менее важно и то, что новая структура удобнее для параллельной работы. Можно отдельно вести backend-ветку, отдельно развивать мобильный UI, отдельно поддерживать контракты и игровые ядра. Это прямо ускоряет командную разработку и уменьшает число конфликтов при интеграции.

С продуктовой стороны проект получил правильное направление: мобильный UX, серверный авторитет, typed API, явные доменные контуры и база для дальнейшего роста. Это именно тот фундамент, который нужен, если продукт планируется развивать долго и системно.

17. Следующий безопасный шаг развития

После завершённой миграции следующая фаза работ уже выглядит значительно спокойнее. Нам не нужно ещё раз чинить основу проекта, прежде чем двигаться дальше. Можно переходить к прикладным задачам: усиливать валидацию game runs, расширять набор тестов, настраивать release pipeline для iOS и Android, подключать реального OTP-провайдера и улучшать визуальную часть мобильных экранов.

Важно, что каждое из этих направлений теперь имеет своё место в архитектуре. Release-скрипты и env-настройки ложатся в мобильный контур. Миграции и доменная логика развиваются в backend. Контракты обновляются в packages. Это снижает стоимость следующего спринта и делает изменения обозримыми для всей команды.

Именно поэтому текущий результат можно считать не промежуточной черновой сборкой, а инженерной платформой. Дальнейшие шаги уже опираются на понятные границы и не требуют повторного пересмотра базовых решений.

- Добавление реального SMS/OTP-провайдера не потребует ломать auth-архитектуру.
- Доработка анимаций и touch UX происходит внутри уже готового мобильного каркаса.
- Расширение доменной логики возможно в существующих модулях без хаотичного роста связности.

Приложение А. Актуальные backend endpoints

Ниже приведена таблица API-ручек, полученная из экспортированного OpenAPI-описания. Она показывает, что backend уже собран как versioned API с auth, профильными и игровыми сценариями.

Method	Path	Summary
POST	/v1/auth/logout	Logout
POST	/v1/auth/refresh	Refresh Tokens
POST	/v1/auth/request-otp	Request Otp
POST	/v1/auth/verify-otp	Verify Otp
GET	/v1/games/summary	Game Summary
POST	/v1/games/{game_code}/runs	Create Game Run
GET	/v1/leaderboard	Leaderboard
GET	/v1/me	Get Me
GET	/v1/profile	Get Profile
PATCH	/v1/profile/focus-planet	Set Focus Planet
GET	/v1/quests	Get Quests
POST	/v1/quests/{quest_id}/claim	Claim Quest
GET	/v1/referrals	Get Referrals
POST	/v1/referrals	Create Referral Invite
GET	/v1/rewards/ledger	Get Reward Ledger

Приложение В. Инвентарь ключевых файлов

В этом приложении собран ключевой файл-лист по основным зонам репозитория. Он нужен не для полного дублирования дерева, а для фиксации фактически присутствующих компонентов новой архитектуры.

Root and docs

- .gitignore
- apps/api/.env.example
- apps/api/alembic.ini
- apps/api/openapi.json
- apps/api/pyproject.toml
- apps/mobile/app.json
- apps/mobile/App.tsx
- apps/mobile/babel.config.js
- apps/mobile/index.js
- apps/mobile/jest.config.js
- apps/mobile/metro.config.js
- apps/mobile/package.json
- apps/mobile/tsconfig.json
- docker-compose.yml
- docs/generate_implementation_report.py
- docs/MTB_Galaxy_Implementation_Report.pdf
- package-lock.json
- package.json
- README.md

Backend scripts

- apps/api/scripts/export_openapi.py
- apps/api/scripts/seed_dev_data.py

Backend source

- apps/api/src/__init__.py
- apps/api/src/app.py
- apps/api/src/common/__init__.py
- apps/api/src/common/enums.py
- apps/api/src/core/__init__.py
- apps/api/src/core/config.py
- apps/api/src/core/security.py
- apps/api/src/db/__init__.py
- apps/api/src/db/base.py

- apps/api/src/db/migrations/env.py
- apps/api/src/db/migrations/versions/0001_initial.py
- apps/api/src/db/models/__init__.py
- apps/api/src/db/models/auth.py
- apps/api/src/db/models/core.py
- apps/api/src/db/models/games.py
- apps/api/src/db/models/quests.py
- apps/api/src/db/models/referrals.py
- apps/api/src/db/models/rewards.py
- apps/api/src/db/session.py
- apps/api/src/infrastructure/__init__.py
- apps/api/src/infrastructure/cache/__init__.py
- apps/api/src/infrastructure/cache/otp_store.py
- apps/api/src/infrastructure/jobs/__init__.py
- apps/api/src/infrastructure/jobs/dispatcher.py
- apps/api/src/main.py
- apps/api/src/modules/__init__.py
- apps/api/src/modules/auth/__init__.py
- apps/api/src/modules/auth/router.py
- apps/api/src/modules/auth/schemas.py
- apps/api/src/modules/auth/service.py
- apps/api/src/modules/devtools/__init__.py
- apps/api/src/modules/devtools/seed.py
- apps/api/src/modules/games/__init__.py
- apps/api/src/modules/games/router.py
- apps/api/src/modules/games/schemas.py
- apps/api/src/modules/games/service.py
- apps/api/src/modules/leaderboard/__init__.py
- apps/api/src/modules/leaderboard/router.py
- apps/api/src/modules/leaderboard/service.py
- apps/api/src/modules/profile/__init__.py
- apps/api/src/modules/profile/router.py
- apps/api/src/modules/profile/schemas.py
- apps/api/src/modules/profile/service.py
- apps/api/src/modules/progression/__init__.py
- apps/api/src/modules/progression/service.py
- apps/api/src/modules/quests/__init__.py
- apps/api/src/modules/quests/router.py
- apps/api/src/modules/quests/service.py

- apps/api/src/modules/referrals/__init__.py
- apps/api/src/modules/referrals/router.py
- apps/api/src/modules/referrals/schemas.py
- apps/api/src/modules/referrals/service.py
- apps/api/src/modules/rewards/__init__.py
- apps/api/src/modules/rewards/router.py
- apps/api/src/modules/rewards/service.py
- apps/api/src/modules/users/__init__.py
- apps/api/src/modules/users/router.py

Backend tests

- apps/api/tests/conftest.py
- apps/api/tests/test_auth.py
- apps/api/tests/test_games.py
- apps/api/tests/test_referrals.py

Mobile source

- apps/mobile/src/app/App.tsx
- apps/mobile/src/app/providers/AppProviders.tsx
- apps/mobile/src/features/auth/screens/SignInScreen.test.tsx
- apps/mobile/src/features/auth/screens/SignInScreen.tsx
- apps/mobile/src/features/games/screens/GamesHubScreen.tsx
- apps/mobile/src/features/games/screens/ShieldGameScreen.tsx
- apps/mobile/src/features/games/screens/SnakeGameScreen.tsx
- apps/mobile/src/features/games/screens/SocialGameScreen.tsx
- apps/mobile/src/features/leaderboard/screens/LeaderboardScreen.tsx
- apps/mobile/src/features/profile/screens/ProfileScreen.tsx
- apps/mobile/src/features/quests/screens/QuestsScreen.tsx
- apps/mobile/src/features/referrals/screens/ReferralsScreen.tsx
- apps/mobile/src/features/rewards/screens/RewardsScreen.tsx
- apps/mobile/src/navigation/RootNavigator.tsx
- apps/mobile/src/shared/api/client.ts
- apps/mobile/src/shared/config/env.ts
- apps/mobile/src/shared/hooks/useSessionBootstrap.ts
- apps/mobile/src/shared/state/session-store.ts
- apps/mobile/src/shared/storage/preferences.ts
- apps/mobile/src/shared/storage/secure-session.ts
- apps/mobile/src/shared/theme/colors.ts
- apps/mobile/src/shared/ui/LoadingView.tsx
- apps/mobile/src/shared/ui/PrimaryButton.tsx

- apps/mobile/src/shared/ui/Screen.tsx
- apps/mobile/src/shared/ui/SecondaryButton.tsx
- apps/mobile/src/shared/ui/SectionCard.tsx
- apps/mobile/src/shared/ui/StatCard.tsx
- apps/mobile/src/shared/ui/TextField.tsx

Shared packages

- packages/contracts/package.json
- packages/contracts/src/generated/api.ts
- packages/contracts/src/index.ts
- packages/game-core/package.json
- packages/game-core/src/index.ts
- packages/game-core/src/shield.ts
- packages/game-core/src/snake.ts
- packages/game-core/src/social.ts
- packages/shared/package.json
- packages/shared/src/index.ts

Приложение С. Технические акценты миграции

Ниже перечислены инженерные акценты, которые уже отражены в репозитории и формируют его текущую зрелость. Этот блок фиксирует не список желаний, а фактические опорные элементы архитектуры, появившиеся после миграции.

- Приложение разделено на чёткие контуры `apps` и `packages`, что упрощает сборку и командную разработку.
- Backend переехал на async FastAPI-стек и перестал зависеть от старой смешанной структуры.
- Схема данных и миграции для PostgreSQL организованы единообразно и готовы к наращиванию.
- OTP и session flow разнесены по ответственным слоям, а не защиты в UI и query-параметры.
- Mobile-клиент подготовлен как отдельное React Native приложение под iOS и Android.
- Серверное состояние и клиентское ephemeral state разведены по разным механизмам хранения.
- Игровые сценарии получили правильное разделение между UX на клиенте и авторитетной фиксацией на сервере.
- Общие контракты и game-core пакеты снижают риск расхождения типов и бизнес-правил.
- Тестовый и скриптовый контур уже присутствует и может расширяться без реорганизации репозитория.
- Текущее состояние проекта позволяет параллельно вести frontend- и backend-разработку отдельными ветками.

В совокупности эти акценты показывают, что мы не просто перенесли файлы по новым папкам, а действительно изменили форму системы. Архитектура стала ближе к production-качеству: доменные решения вынесены на сервер, мобильный клиент получил правильные границы ответственности, а общие пакеты закрепили единый язык взаимодействия между слоями.